

随机数生成问题分析

1. 基本情况

- 目前随机数生成用的是 Philox/counter-based RNG 算法, 底层调用 murand_normal;
- 该算法是比较固定的一种生成方式, state 一致的化生成的随机数和 GPU 架构无关, 关键是 state 的生成.

DistributionTemplates.h

```
template<typename scalar_t, typename accscalar_t, typename RNG, typename transform_t>
void normal_and_transform(TensorIteratorBase& iter, RNG gen, transform_t transform) {
    if (std::is_same<scalar_t, double>::value) {
        distribution_nullary_kernel<scalar_t, accscalar_t, double2>(iter,
            gen,
            [] __device__ (murand_state_philox4x32_10* state) -> double2 { return murand_normal2_double(state); },
            transform);
    } else {
        distribution_nullary_kernel<scalar_t, accscalar_t, float4>(iter,
            gen,
            [] __device__ (murand_state_philox4x32_10* state) -> float4 { return murand_normal4(state); },
            transform);
    }
}
```

2. state 的确定

- murand_init 用 seed 和 idx 来确定 state;
- state 直接影响随机数生成;
- idx 由grid 由grid size 和 block size 确定;
- block size 不同架构一致, grid size 需要看情况.
- 结论: grid dim 是最终决定不同架构随机数生成的唯一不同因素.

DistributionTemplates.h

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
murand_state_philox4x32_10 state;
murand_init(std::get<0>(seeds),
            idx,
            std::get<1>(seeds),
            &state);
```

3. grid dim 确定:

- maxThreadsPerMultiProcessor 和 multiProcessorCount 是影响因素;
- 以上两者和GPU 架构有关, 会影响到随机数生成.

DistributionTemplates.h

```
std::tuple<uint64_t, dim3, dim3> calc_execution_policy(const int64_t total_elements, const uint32_t
unroll_factor) {
    const uint64_t numel = static_cast<uint64_t>(total_elements);
    const uint32_t block_size = block_size_bound;
    dim3 dim_block(block_size);
    dim3 grid((numel + block_size - 1) / block_size);
    uint32_t blocks_per_sm = at::musa::getCurrentDeviceProperties()->maxThreadsPerMultiProcessor / block_size;
    grid.x = std::min(
        static_cast<uint32_t>(at::musa::getCurrentDeviceProperties()->multiProcessorCount) * blocks_per_sm,
        grid.x);
    //number of times random will be generated per thread, to offset philox counter in the random state
    uint64_t counter_offset = ((numel - 1) / (block_size * grid.x * unroll_factor) + 1) *
max_generator_offsets_per_curand_call;
    return std::make_tuple(counter_offset, grid, dim_block);
}
```

4. 结论:

- `tensor.numel > 184320` 后, S5000 生成的随机数和 A100/H100 生成随机数不一致; A100和H100也不一致。
- `tensor.numel ≤ 184320` 时, S5000 生成的随机数和 A100/H100 等生成随机数完全相同。

5. 注意事项

- 一旦有一个随机数生成不一致, torch RNG 状态被破坏, 之后的随机数生成均不一致。
- 当 `tensor` 元素个数刚超过 184320 时, 不如 184321 大部分生成的随机数是一致的, 最末尾的随机数先开始不一致;
- 继续生成时, 从后向前不一致现象逐步扩散。

6. 工程实践: 如何解决cosyvoice flow 模型中的随机数引起的 loss 不对齐

6.1. cosyvoice flow 随机数

```
streaming = True if random.random() < 0.5 else False

# random timestep
t = torch.rand([b, 1, 1], device=mu.device, dtype=mu.dtype)

# sample noise p(x_0) : 15000~18000
z = torch.randn_like(x1)

cfg_mask = torch.rand(b, device=x1.device) > self.training_cfg_rate

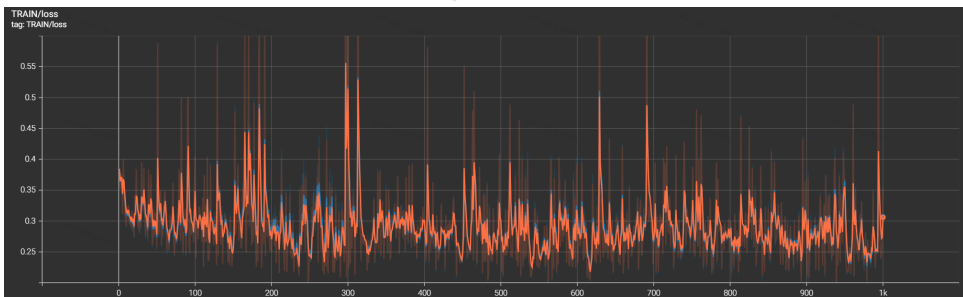
# sdpa dropout 0
x = F.scaled_dot_product_attention(query, key, value, attn_mask=attn_mask, dropout_p=0.0, is_causal=False)

# attn.to_out : tensor numel > 200000
self.to_out.append(nn.Dropout(dropout))

# mlp : tensor numel > 200000
self.fff = nn.Sequential(project_in, nn.Dropout(dropout), nn.Linear(inner_dim, dim_out))
```

6.2. 优化效果

- 设置dropout 的dropout_p hard code 为0;
- 增加判断条件 : `batch_dict['speech_feat'].numel() > 184320` 时, 跳过该step;
- 8卡效果:



具体值:

```
cuda_loss_count: 2001
musa_loss_count: 2001
matched_step_count: 2001
average_absolute_error: 0.00431019790105
average_relative_error: 0.0148611280008
```